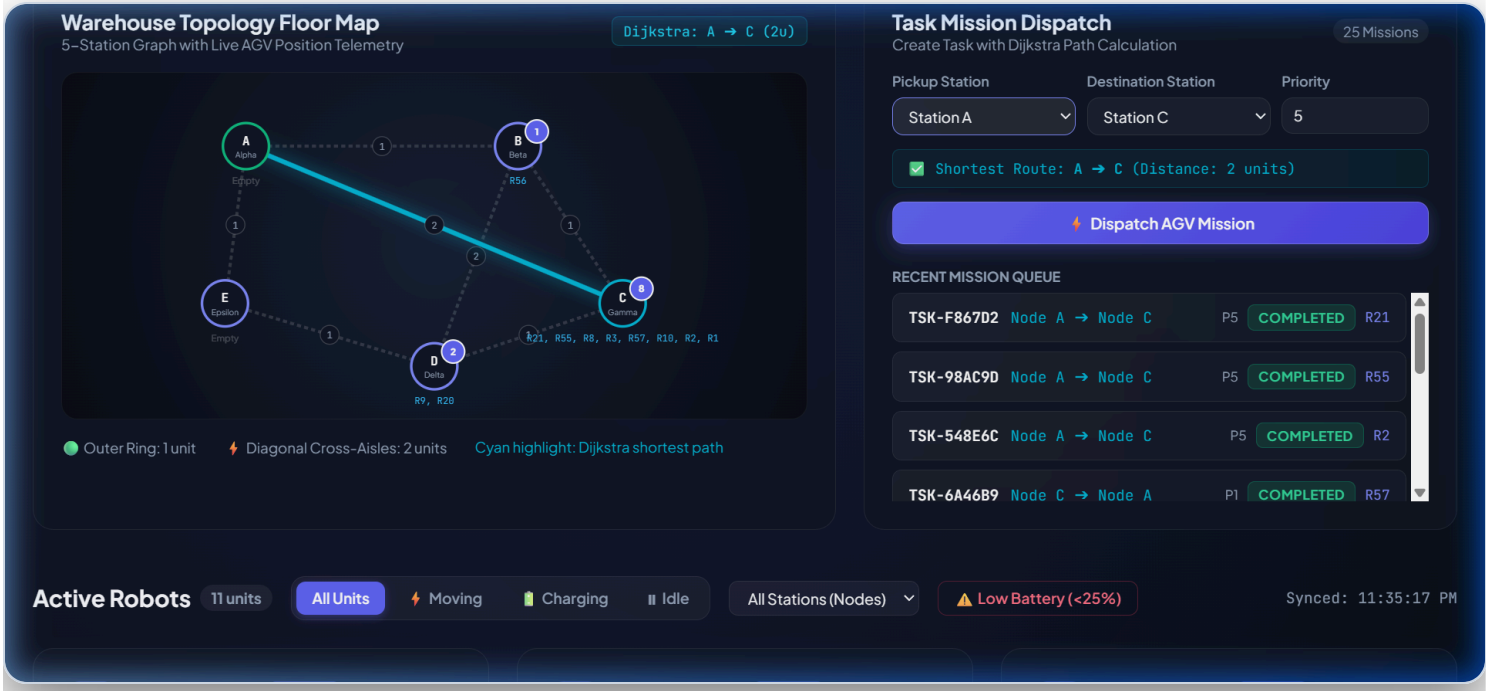
Robot Fleet Manager

Complete Engineering Architecture, Implementation Phases & Study Guide

📸 1. Live Working Web Dashboard

The interactive dashboard runs live on `http://localhost:8080/`, providing real-time SVG floor map tracking, Dijkstra route previews, priority mission queues, and AGV fleet statuses:



🚩 2. Project Genesis: Where It Started

Baseline Setup Before Redesign:

- **Database Table:** Single PostgreSQL table `robots` with columns `(id, current_node, battery_percent, status)`.
- **Hardcoded Movement:** `RobotMonitorService.java` moved all robots continuously through an endless hardcoded array:

```
```

- **No Topology:** Corridors had no distances, no obstacles, and no track routing.
- **No Missions:** Robots had no customer tasks or delivery destinations.

🚀 3. The 5 Evolutionary Phases

PHASE 1 Task Management System

Goal: Transform random movement into customer-requested, prioritized missions.

- **JPA Entity:** `Task.java` with fields `id` (e.g. `TSK-ED9908`), `pickupNode`, `destinationNode`, `priority` (1–10), `status`, and `assignedRobotId`.
- **State Machine (`TaskStatus.java`):** `CREATED` ➔ `ASSIGNED` ➔ `RUNNING` ➔ `COMPLETED` / `FAILED`.
- **REST API:** `POST /api/tasks` (Create mission) and `GET /api/tasks` (List queue ordered by priority DESC).

PHASE 2 Warehouse Graph Topology & Dijkstra Router

Goal: Provide realistic spatial corridors and shortest path route optimization.

- **Graph Model (`WarehouseGraph.java`):**
  - 5 Stations: A (Alpha), B (Beta), C (Gamma), D (Delta), E (Epsilon).
  - Perimeter Single Corridors (Weight = 1): A-B, B-C, C-D, D-E, E-A.
  - Cross-Aisle Diagonal Expressways (Weight = 2): B-D, A-C.
- **Dijkstra Algorithm (`RouterService.java`):**
  - Calculates mathematical shortest path using `PriorityQueue<NodeDistance>` .
  - Example: Route A ➔ C calculates direct diagonal `[A, C]` with cost = 2.
- **Interactive SVG Map:** Added to `index.html` with glowing route highlights and station click listeners.

PHASE 3 & 4 Dispatcher Engine & Autonomous Navigation

Goal: Match eligible idle robots to missions and guide them step-by-step.

- **Dispatcher Engine (`DispatcherService.java`):**
  - Selects highest priority `CREATED` tasks.
  - Filters available robots: `status == IDLE` and `battery ≥ 25%` .
  - Prevents double assignment by atomically locking robot to task.
- **Mission Navigation Loop (`RobotMonitorService.java`):**
  1. Robot at Start Node ➔ Navigates Dijkstra route to **Pickup Station**.
  2. Arrives at Pickup ➔ Task transitions to `RUNNING` .
  3. Navigates Dijkstra route from Pickup to **Destination Station**.
  4. Arrives at Destination ➔ Task transitions to `COMPLETED` .
  5. Robot halts and transitions back to `IDLE` standby.

PHASE 5 Exclusive Resource Scheduling & Deadlock Prevention

Goal: Guarantee zero physical collisions and circular deadlocks on single-lane corridors.

- **Exclusive Station Locks:** Only one robot can occupy a station node at a time (`tryReserveNode`).
- **Canonical Single-Lane Edge Locks:** Corridors are shared tracks. To prevent head-on crashes, keys are canonically sorted:

  Robot entering A ➔ B locks `"A-B"` , blocking opposing robot from entering B ➔ A.
- **Deadlock Counter & Detour Bypasses:**
  - When blocked, robot yields (status: `WAITING`) and increments `waitCounters` .
  - At ≥ 2 wait cycles, `RouterService.findShortestPathAvoidingNodes` calculates a dynamic detour bypass around the blocked node, breaking circular waits!

💻 4. Command Line Manual (Cheat Sheet)

1. Build and Run Tests

2. REST API cURL Commands

3. PostgreSQL Direct Inspection

✅ 5. Test Suite Verification Matrix (18/18 Passing)

Test Class	Tests	Result	Verified Behaviors
TaskServiceTests	4	PASSED	Task creation, UUID prefix, status transitions, priority queue ordering
RouterServiceTests	5	PASSED	Dijkstra shortest paths, edge costs, detour paths avoiding blocked stations
DispatcherServiceTests	4	PASSED	Idle robot matching, battery ≥ 25% check, double assignment prevention
ResourceSchedulerTests	4	PASSED	Exclusive station locks, head-on corridor locking, deadlock wait

Test Class	Tests	Result	Verified Behaviors
			counter, bypass routing
`FleetmanagerApplicationTests`	1	PASSED	Spring Boot full application context load

Autonomous Robot Fleet Manager — Complete Engineering Study Guide © 2026.